Project RM: Mathematical Architecture & Quantitative Model Specification

Author: Quantitative Research & Architecture Team
System: Project RM (Risk Management & Alpha Engine)
Classification: Institutional Mathematical Specification
Version: 3.4.0-Production

1. Executive Summary & Architectural Overview

Project RM is an institutional-grade quantitative derivative trading and risk management platform. The system operates on the core empirical premise that idiosyncratic single-stock returns, when properly orthogonalized against broad market indices and sector benchmark Exchange-Traded Funds (ETFs), exhibit localized structural mean-reverting stationary dynamics.

By measuring the standardized residual innovation (ZZ-score) of an asset relative to a continuously updating state-space dynamic beta model, the system systematically identifies statistical extremes, models their mean-reversion trajectories, filters out uncompensated binary corporate event risks (earnings announcements), and executes directional At-The-Money (ATM) option contracts optimized over a 3-dimensional parameter manifold: **Trigger Barrier (σ^* σ∗)**, **Contract Tenor (τ^* τ∗)**, and **Holding Horizon (H^* H∗)**.

The computational pipeline consists of seven dedicated modules (pages) backed by an ACID-compliant local SQL data warehouse and real-time streaming/REST market data connectors:

2. Page-by-Page Model Summary & Operational Role

The front-end user interface is partitioned into seven distinct modules, each responsible for a specific phase of quantitative inference, parameter calibration, risk screening, and trade execution.

2.1. Dynamic Edge (`cockpit`)

- **Core Purpose:** The central interactive quantitative cockpit for real-time asset analysis, model parameterization, live signal monitoring, and execution blueprint synthesis.

- **Models Deployed:**
    1. *Dual-State Dynamic Kalman Filter:* Continuously estimates the time-varying sensitivity of the target asset y_t yt to the market factor $r_{M,t}$ rM,t and orthogonal sector factor $r_{S \perp M,t}$ rS⊥M,t.
    2. *Ornstein-Uhlenbeck (OU) Mean-Reversion Monitor:* Solves for the annualized rate of mean reversion θθ, equilibrium level μμ, and residual half-life $t_{1/2}$ t1/2.
    3. *Page's Cumulative Sum (CUSUM) Drift Detector:* Tracks persistent directional drift (S_t^+, S_t^- St+, St−) across consecutive daily residual innovations to alert against structural factor decoupling.
    4. *Unified Trade Execution Blueprint:* Computes option contract requirements, optimal entry thresholds, recommended DTE, target hold periods, and Black-Scholes theoretical entry valuations.

2.2. Rare Edge (`gold-ledger`)

- **Core Purpose:** The institutional champion registry tracking the top-performing, walk-forward-validated parameter triplets across the curated watchlist ($N = 21$N = 21 liquid blue-chip equities).
- **Models Deployed:**
    1. *3D Champion Triplet Registry:* Stores the globally optimal (σ^*, τ^*, H^*)(σ∗,τ∗,H∗) discovered through exhaustive surface optimization.
    2. *Pre-Close 3:45 PM Market-On-Close (MOC) Screener:* Scans all 21 core equities between 3:45 PM and 3:58 PM EST. Evaluates instantaneous innovation ZZ-scores against the asset's specific champion barrier σ^* σ∗.
    3. *Order Specification Generator:* Instantly generates trade directives (`BUY ATM CALL` or `BUY ATM PUT`) with target strike, contract tenor, and exit session count when $|Z_t| \geq \sigma^*$ |Zt| ≥ σ∗.

2.3. Tri-Surface Edge (`tenor-sweep`)

- **Core Purpose:** Visualizes and interrogates the 3-dimensional optimization manifold $S(\sigma, \tau, H)$ S(σ,τ,H) over contract maturities ($\tau \in \{7, 10, 14, 21, 35, 45, 60, 90\}$τ ∈ {7, 10, 14, 21, 35, 45, 60, 90} days), trigger barriers ($\sigma \in [1.0, 3.0]$σ ∈ [1.0, 3.0]), and holding sessions ($H \in [1, 21]$H ∈ [1, 21]).
- **Models Deployed:**
    1. *2D Slice Parameter Grid:* Produces interactive heatmaps for four performance metrics: Profit Factor (PFPF), Win Rate ($WR\%$WR%), Average Return on Investment ($ROI\%$ROI%), and Total Dollar P&L.
    2. *Plateau Smoothing Regularization:* Computes $3 \times 3$3 × 3 local neighborhood utility averages to eliminate overfitted local point spikes in favor of robust parameter plateaus.

3. *65/35 Chronological Walk-Forward Validator:* Splits the historical time-series into In-Sample (IS: 65%) and Out-of-Sample (OOS: 35%) sets to compute the empirical Efficiency Ratio $\eta = WR_{\text{OOS}} / WR_{\text{IS}}$ η = WROOS/WRIS.

2.4. Cadence Edge (`income-strategy`)

- **Core Purpose:** High-velocity cashflow engine designed for recurring weekly income generation, focusing on shorter turnaround holding periods (1 to 5 sessions) and high trade arrival rates.
- **Models Deployed:**
  1. *Arrival Rate Poisson / Renewal Process:* Models annual trade count N_{trades} Ntrades as an arrival velocity $\lambda = N_{\text{trades}} / 52$λ = Ntrades/52 trades/week.
  2. *Proximity Radar Screener:* Operates between 3:45 PM and 4:00 PM EST, ranking all 24 calibrated setups by standard deviation proximity $\delta_\sigma = \max(0.0, \sigma^* - |Z_t|)$δσ = max(0.0, σ∗ − |Zt|) to their trigger barrier.
  3. *Blended Cashflow Portfolio Simulator:* Dynamically aggregates selected subsets of income setups into a simulated multi-asset basket, calculating weighted win rate, weighted holding period, and portfolio blended profit factor.

2.5. Atlas Edge (`price-action`)

- **Core Purpose:** Multi-timeframe technical price action and Kalman trajectory verification.
- **Models Deployed:**
  1. *Kalman Orthogonal Reconstructed Price Channel:* Overlays the predicted underlying price trajectory $\hat{y}_t$ y^t derived from market and sector factor pass-through against actual price action.
  2. *Technical Confluence Geometry:* Identifies structural horizontal support/resistance levels, rolling swing high/low bounds, and moving average benchmarks in alignment with residual trigger breaches.

2.6. Dossier Edge (`trades-log`)

- **Core Purpose:** Forensic execution audit and historical walk-forward trade journal.
- **Models Deployed:**
  1. *Simulated ATM Option Trade Execution Engine:* Chronologically simulates historical trade entries at closing prices, tracking price path evolution session-by-session through exit at horizon H^* H∗.
  2. *Corporate Earnings Announcement Blackout Filter:* Detects whether an entry date falls on an earnings announcement date or the immediate subsequent trading session (

$t \in \{t_{\text{earn}}, t_{\text{earn}} + 1\}$t ∈ {tearn, tearn + 1}). Gating out binary earnings risk protects options positions from implied volatility crush and unhedged gap risk.

3. *Forensic Trade Attribution:* Classifies losing trades into structural failure categories (`REGIME_DRIFT`, `THETA_BLEED`, or `WHIPSAW`).

2.7. Diagnostic Edge (`diagnostics`)

- **Core Purpose:** Econometric and statistical hypothesis testing laboratory for model validation and assumption verification.
- **Models Deployed:**
  1. *Augmented Dickey-Fuller (ADF) & Phillips-Perron Stationarity Tests:* Formally tests the residual spread for unit roots to confirm mean-reverting cointegration.
  2. *Ljung-Box Innovation Whiteness Test:* Tests whether Kalman filter residuals e_t et are statistically indistinguishable from white noise (QQ-statistic).
  3. *Discrete Semi-Markov Regime Duration Model:* Identifies state transition matrices and estimates dwell-time distributions (Geometric vs. Discrete Weibull) to quantify regime persistence and hazard rates.

3. Mathematical Formulations & Derivations

This section provides complete mathematical formulations, state-space equations, optimization objectives, and derivation steps necessary for independent quantitative audit.

```
======================================================================
                MATHEMATICAL SPECIFICATION INDEX
======================================================================
[3.1] Dual-Factor Orthogonal State-Space Kalman Filter (QMLE)
[3.2] Continuous-Time Ornstein-Uhlenbeck (OU) Mean-Reversion SDE
[3.3] Two-Sided Page's CUSUM Stealth Drift & Structural Break Model
[3.4] Discrete Semi-Markov Volatility Regime & Duration Hazard Model
[3.5] 3D Parameter Optimization Manifold & Plateau Regularization
[3.6] Closed-Form Vectorized Black-Scholes-Merton ATM Option Engine
[3.7] Cadence Edge Signal Arrival & Portfolio Velocity Dynamics
======================================================================
```

3.1. Dual-Factor Orthogonal State-Space Kalman Filter

3.1.1. Factor Orthogonalization (Gram-Schmidt)

Let $y_t = \ln(P_{\text{asset},t}/P_{\text{asset},t-1})$yt = ln(Passet,t/Passet,t−1) denote the daily log-return of the target equity.
Let $r_{M,t} = \ln(P_{\text{market},t}/P_{\text{market},t-1})$rM,t = ln(Pmarket,t/Pmarket,t−1) denote the log-return of the broad market benchmark (e.g., SPY, QQQ, DIA).
Let $r_{S,t} = \ln(P_{\text{sector},t}/P_{\text{sector},t-1})$rS,t = ln(Psector,t/Psector,t−1) denote the raw log-return of the sector ETF (e.g., XLK, SMH, XLE).

To eliminate collinearity between the sector ETF and broad market index, $r_{S,t}$ rS,t is strictly orthogonalized with respect to $r_{M,t}$ rM,t:

$$\gamma = \frac{\sum_{t=1}^{T}(r_{M,t}-\bar{r}_M)(r_{S,t}-\bar{r}_S)}{\sum_{t=1}^{T}(r_{M,t}-\bar{r}_M)^2} = \frac{\text{Cov}(r_M, r_S)}{\text{Var}(r_M)}$$

γ = ∑t=1T(rM,t − r¯M)2∑t=1T(rM,t − r¯M)(rS,t − r¯S) = Var(rM)Cov(rM, rS)

$$r_{S\perp M,t} = r_{S,t} - \gamma r_{M,t}$$

rS⊥M,t = rS,t − γ rM,t

The observation regressor row vector at time tt is:

$$X_t = [\ 1 \quad r_{M,t} \quad r_{S\perp M,t}\ ] \in R^{1\times 3}$$

Xt = [1rM,trS⊥M,t] ∈ R1×3

3.1.2. State-Space Representation

The dynamic linear model is formulated as:

$$\begin{aligned} &\text{State Equation:} && \beta_t = T_t\beta_{t-1} + w_t, && w_t \sim N(0,Q) \\ &\text{Observation Equation:} && y_t = X_t\beta_t + v_t, && v_t \sim N(0,R) \end{aligned}$$

State Equation: Observation Equation: βt = Ttβt−1 + wt, yt = Xtβt + vt, wt ∼ N(0, Q)vt ∼ N(0, R)

where:

- $\beta_t = [\ \alpha_t \quad \beta_{\text{market},t} \quad \beta_{\text{sector},t}\ ]^T \in R^{3\times 1}$ βt = [αtβmarket,tβsector,t]T ∈ R3×1 is the unobserved state vector.
- $T_t = I_3$ Tt = I3 (random walk transition matrix).
- $Q = \text{diag}(q_\alpha, q_{\text{market}}, q_{\text{sector}}) \in R^{3\times 3}$ Q = diag(qα, qmarket, qsector) ∈ R3×3 is the process noise covariance matrix.
- $R \in R^+$ R ∈ R+ is the scalar observation noise variance.
- $E[w_t\, v_s] = 0 \quad \forall t, s$E[wtvs] = 0 ∀t, s.

3.1.3. Recursive Filter Equations

Given the prior state estimate $\hat{\beta}_{t-1|t-1}$ β^t−1|t−1 and error covariance $P_{t-1|t-1}$ Pt−1|t−1:

1. State Prediction: $\hat{\beta}_{t|t-1} = \hat{\beta}_{t-1|t-1}$ β^t|t−1 = β^t−1|t−1

2. Covariance Prediction: $P_{t|t-1} = P_{t-1|t-1} + Q$ Pt|t−1 = Pt−1|t−1 + Q

3. Measurement Innovation (Residual): $\hat{y}_t = X_t \hat{\beta}_{t|t-1}$ y^t = Xtβ^t|t−1 $e_t = y_t - \hat{y}_t = y_t - X_t \hat{\beta}_{t|t-1}$ et = yt − y^t = yt − Xtβ^t|t−1

4. Innovation Variance (S_t St): $S_t = X_t P_{t|t-1} X_t^T + R \in R^+$ St = XtPt|t−1XtT + R ∈ R+

5. Optimal Kalman Gain Matrix (K_t Kt): $K_t = \frac{P_{t|t-1} X_t^T}{S_t} \in R^{3\times 1}$ Kt = StPt|t−1XtT ∈ R3×1

6. State Posterior Update: $\hat{\beta}_{t|t} = \hat{\beta}_{t|t-1} + K_t e_t$ β^t|t = β^t|t−1 + Ktet

7. Error Covariance Posterior Update: $P_{t|t} = (I_3 - K_t X_t) P_{t|t-1}$ Pt|t = (I3 − KtXt)Pt|t−1

3.1.4. Standardized Innovation ZZ-Score

The standardized 1-step-ahead residual innovation Z_t Zt is defined as:

$$Z_t = \frac{e_t}{\sqrt{S_t}} = \frac{y_t - X_t \hat{\beta}_{t|t-1}}{\sqrt{X_t P_{t|t-1} X_t^T + R}}$$

Zt = St

---

et = XtPt|t−1XtT + R

yt − Xtβ^t|t−1

Under nominal model calibration, $Z_t \overset{\text{iid}}{\sim} N(0,1)$ Zt ∼iid N (0, 1).

3.1.5. Quasi-Maximum Likelihood Estimation (QMLE)

The hyperparameter vector $\psi = [\ q_1 \quad q_2 \quad q_3 \quad R\ ]^T$ ψ = [q1q2q3R]T is optimized by minimizing the negative log-likelihood function via the L-BFGS-B algorithm:

$$L(\psi) = \frac{1}{2} \sum_{t=1}^{T} (\ln(2\pi) + \ln(S_t(\psi)) + \frac{e_t(\psi)^2}{S_t(\psi)})$$

L(ψ) = 21 t=1∑T (ln(2π) + ln(St(ψ)) + St(ψ)et(ψ)2)

$$\psi^* = \arg\min_{\psi} L(\psi) \quad \text{subject to } 10^{-8} \le q_i \le 10^{-2},\ 10^{-6} \le R \le 3 \cdot \mathrm{Var}(y)$$

ψ∗ = arg ψmin L(ψ) subject to 10−8 ≤ qi ≤ 10−2, 10−6 ≤ R ≤ 3 · Var(y)

3.2. Continuous-Time Ornstein-Uhlenbeck (OU) Mean-Reversion SDE

The continuous-time dynamics of the standardized residual spread Z_t Zt are governed by the stochastic differential equation:

$$dZ_t = \theta(\mu - Z_t)dt + \sigma_{OU}\, dW_t$$

dZt = θ(μ − Zt)dt + σOU dWt

where:

- $\theta > 0$θ > 0 is the mean-reversion speed parameter.
- μμ is the long-term equilibrium level (asymptotically $\mu \to 0$μ → 0).
- $\sigma_{OU} > 0$σOU > 0 is the diffusion volatility.
- W_t Wt is a standard 1-dimensional Brownian motion ($E[dW_t] = 0, E[(dW_t)^2] = dt$E[dWt] = 0, E[(dWt)2] = dt).

3.2.1. Exact Discrete-Time AR(1) Formulation

Integrating the SDE over discrete sampling intervals $\Delta t = 1$Δt = 1 trading day yields an exact autoregressive representation:

$$Z_t = a + bZ_{t-1} + \epsilon_t, \quad \epsilon_t \sim N(0, \sigma_\epsilon^2)$$

Zt = a + b Zt−1 + ϵt, ϵt ∼ N (0, σϵ2)

where coefficients map directly to continuous parameters via:

$$b = e^{-\theta\Delta t} \implies \theta = -\frac{\ln(b)}{\Delta t}$$

b = e−θΔt ⟹ θ = −Δtln(b)

$$a = \mu(1 - e^{-\theta\Delta t}) \implies \mu = \frac{a}{1-b}$$

a = μ(1 − e−θΔt) ⟹ μ = 1 − ba

$$\sigma_\epsilon^2 = \frac{\sigma_{OU}^2}{2\theta}(1 - e^{-2\theta\Delta t}) = \frac{\sigma_{OU}^2}{2\theta}(1 - b^2)$$

σϵ2 = 2θσOU 2 (1 − e−2θΔt) = 2θσOU 2(1 − b2)

3.2.2. Half-Life Derivation

The half-life $t_{1/2}$ t1/2 represents the expected time required for the expected deviation $E[Z_t - \mu]$E[Zt − μ] to decay by half:

$$E[Z_{t+\tau} - \mu \mid Z_t] = (Z_t - \mu)e^{-\theta\tau}$$

E[Zt+τ − μ ∣ Zt] = (Zt − μ)e−θτ

Setting $e^{-\theta t_{1/2}} = \frac{1}{2}$e−θt1/2 = 21:

$$-\theta t_{1/2} = \ln(\frac{1}{2}) = -\ln(2) \implies t_{1/2} = \frac{\ln(2)}{\theta} = -\frac{\ln(2)\cdot\Delta t}{\ln(b)}$$

−θt1/2 = ln (21) = − ln(2) ⟹ t1/2 = θln(2) = −ln(b)ln(2) ⋅ Δt

3.2.3. Stationarity Criterion

A valid mean-reverting process requires strict bounded stability:

$$0 < b < 1 \iff \theta > 0$$

0 < b < 1 ⟺ θ > 0

If $b \geq 1.0$b ≥ 1.0 or $b \leq 0.0$b ≤ 0.0, the process exhibits non-stationary unit root or oscillatory divergence, and the mean-reversion flag is set to `False`.

3.3. Two-Sided Page's CUSUM Stealth Drift & Structural Break Model

To detect subtle structural factor drift that does not trigger instantaneous barrier breaches ($|Z_t| \geq 2\sigma$|Zt| ≥ 2σ) but exhibits persistent directional bias, the system runs Page's two-sided Cumulative Sum (CUSUM) test.

3.3.1. Recursive Accumulators

With reference allowance $k = 0.5$k = 0.5 (tolerance parameter) and decision threshold $h = 4.0$h = 4.0:

$$\begin{aligned} S_0^+ &= 0, \quad S_t^+ = \max(0, S_{t-1}^+ + Z_t - k) \quad \text{[Bullish / Positive Factor Drift]} \\ S_0^- &= 0, \quad S_t^- = \max(0, S_{t-1}^- - Z_t - k) \quad \text{[Bearish / Negative Factor Drift]} \end{aligned}$$

S0+S0− = 0, = 0, St+ = max(0, St−1+ + Zt − k) St− = max(0, St−1− − Zt − k)
[Bullish / Positive Factor Drift][Bearish / Negative Factor Drift]

3.3.2. Structural Alert Logic

A structural regime shift alert is issued at time tt if either cumulative accumulator breaches threshold h h:

$$\text{Regime Alert}_t = \begin{cases} +1 & \text{(Stealth Bullish Breakout)}, \quad \text{if } S_t^+ \geq h \\ -1 & \text{(Stealth Bearish Breakdown)}, \quad \text{if } S_t^- \geq h \\ 0 & \text{(Stationary Mean-Reverting)}, \quad \text{otherwise} \end{cases}$$

Regime Alertt = ⎩ ⎭ ⎧ +1 (Stealth Bullish Breakout),
−1 (Stealth Bearish Breakdown),0 (Stationary Mean-Reverting),if St+ ≥ hif St− ≥ hotherwise

3.4. Discrete Semi-Markov Volatility Regime & Duration Hazard Model

Unlike classical Markov chains where sojourn durations are strictly constrained to memoryless Geometric distributions, Project RM deploys a **Discrete Semi-Markov Model (SMM)** allowing state dwell times to follow duration-dependent hazard functions.

3.4.1. State Space Discretization

The continuous innovation series Z_t Zt is discretized into three orthogonal operating regimes:

$$S_t = \begin{cases} 0 & \text{UNDERPERFORMING}, \quad \text{if } Z_t \le -\sigma^* \\ 1 & \text{NEUTRAL}, \quad \text{if } -\sigma^* < Z_t < +\sigma^* \\ 2 & \text{OUTPERFORMING}, \quad \text{if } Z_t \ge +\sigma^* \end{cases}$$

St = ⎩⎨⎧0 UNDERPERFORMING,1 NEUTRAL,2 OUTPERFORMING,if Zt ≤ −σ∗if −σ∗ < Zt < +σ∗if Zt ≥ +σ∗

3.4.2. Embedded Markov Transition Matrix

Let $J_n \in \{0, 1, 2\}$Jn ∈ {0, 1, 2} denote the state visited during the nn-th regime transition, with T_n Tn representing the transition timestamp. The embedded transition probability matrix $P = [p_{ij}]$P = [pij] satisfies:

$$p_{ij} = P(J_{n+1} = j \mid J_n = i), \quad p_{ii} = 0, \quad \sum_{j \neq i} p_{ij} = 1$$

pij = P(Jn+1 = j ∣ Jn = i), pii = 0, j□=i∑ pij = 1

$$\hat{p}_{ij} = \frac{N_{ij}}{\sum_{k \neq i} N_{ik}}$$

p^ij = ∑k□=i NikNij

where N_{ij} Nij is the observed count of direct transitions from regime ii to regime jj.

3.4.3. Dwell-Time Distribution Models

Model A: Geometric Distribution (Memoryless Null Hypothesis)

Probability mass function for duration $t \in \{1, 2, 3, \dots\}$t ∈ {1, 2, 3, ...}:

$$f_{\text{geom}}(t) = (1-p)^{t-1} p, \quad \hat{p} = \frac{1}{\bar{t}} = \frac{1}{\frac{1}{M}\sum_{m=1}^{M} t_m}$$

fgeom(t) = (1 − p)t−1p, p^ = t¯1 = M1 ∑m=1M tm1

- Survival Function: $S_{\text{geom}}(t) = P(T > t) = (1-p)^t$ Sgeom(t) = P(T > t) = (1 − p)t
- Constant Hazard Function: $\lambda(t) = \frac{f(t)}{S(t-1)} = p$λ(t) = S(t−1)f(t) = p

Model B: Discrete Weibull Distribution (Nakagawa & Osaki Parameterization)

To capture increasing or decreasing hazard rates over time, the system fits a Discrete Weibull distribution:

$$S_{\text{weib}}(t) = q^{t^\beta}, \quad q \in (0,1), \beta > 0$$

Sweib(t) = qtβ, q ∈ (0, 1), β > 0

$$f_{\text{weib}}(t) = S_{\text{weib}}(t-1) - S_{\text{weib}}(t) = q^{(t-1)^\beta} - q^{t^\beta}$$

fweib(t) = Sweib(t − 1) − Sweib(t) = q(t−1)β − qtβ

The discrete hazard rate $\lambda(t)$λ(t) is duration-dependent:

$$\lambda(t) = 1 - q^{t^\beta - (t-1)^\beta}$$

λ(t) = 1 − qtβ −(t−1)β

- If $\beta > 1$β > 1: Hazard rate is **increasing** with duration ($\lambda(t) \uparrow$ λ(t) ↑). As a regime persists, probability of exit increases (mean-reverting pressure).
- If $\beta < 1$β < 1: Hazard rate is **decreasing** with duration ($\lambda(t) \downarrow$ λ(t) ↓). Long streaks become progressively more persistent.
- If $\beta = 1$β = 1: Reduces exactly to the memoryless Geometric distribution ($q = 1 - p$q = 1 − p).

Parameters (q, β)(q, β) are estimated via maximum likelihood:

$$\ln L(q, \beta) = \sum_{m=1}^{M} \ln(q^{(t_m - 1)^\beta} - q^{t_m^\beta})$$

ln L(q, β) = m=1∑M ln (q(tm −1)β − qtmβ)

Model Selection: Akaike Information Criterion (AIC)

$$\text{AIC} = 2k - 2\ln\hat{L}$$

AIC = 2k − 2 ln L^

where $k = 1$k = 1 for Geometric and $k = 2$k = 2 for Discrete Weibull. The model with minimum AIC is automatically selected.

3.4.4. Conditional Exit Probability & Mean Residual Life

Given that the current regime has already persisted for NN consecutive trading sessions:

1. Conditional Exit Probability within kk days: $P(\text{Exit} \le N + k \mid T > N) = \frac{S(N) - S(N+k)}{S(N)}$
P(Exit ≤ N + k ∣ T > N) = S(N)S(N)−S(N+k)

2. Mean Residual Life (MRL): $\text{MRL}(N) = E[T - N \mid T > N] = \frac{1}{S(N)} \sum_{t=N+1}^{\infty} S(t)$
MRL(N) = E[T − N ∣ T > N] = S(N)1 ∑t=N+1∞ S(t)

3.5. 3D Parameter Optimization Manifold & Plateau Regularization

The optimization engine identifies the global champion parameter triplet:

$$\omega^* = (\sigma^*, \tau^*, H^*) \in \Omega$$

ω∗ = (σ∗, τ∗, H∗) ∈ Ω

where:

- $\sigma \in \{1.2, 1.4, 1.6, 1.8, 2.0, 2.2, 2.4\}$σ ∈ {1.2, 1.4, 1.6, 1.8, 2.0, 2.2, 2.4} is the entry trigger standard deviation.
- $\tau \in \{7, 10, 14, 21, 35, 45, 60, 90\}$τ ∈ {7, 10, 14, 21, 35, 45, 60, 90} is the option contract tenor (DTE).
- $H \in \{1, 2, 3, 4, 5, 7, 10, 12, 14, 15, 18, 21\}$H ∈ {1, 2, 3, 4, 5, 7, 10, 12, 14, 15, 18, 21} is the trade holding horizon in sessions.

3.5.1. Raw Cell Utility Function

For any parameter pair (σ, H)(σ, H) at contract tenor ττ:

$$U(\sigma, H) = \begin{cases} (\frac{WR}{100}) \times R\bar{O}I \times \sqrt{N_{\text{trades}}}, & \text{if } N_{\text{trades}} \geq N_{\min}, R\bar{O}I > 0, WR \geq 35\% \\ 0, & \text{otherwise} \end{cases}$$

U(σ, H) = {(100WR) × ROI × Ntrades

,0,if Ntrades ≥ Nmin, ROI > 0, WR ≥ 35%otherwise

where:

- WRWR is the empirical win rate percentage.
- $R\bar{O}I$ROI is the average return on investment per trade.
- N_{trades} Ntrades is the sample trade count.

3.5.2. Plateau Smoothing Regularization

To penalize isolated parameter spikes (overfitting) and reward broad, stable parameter regions, a 3×3 3 × 3 local neighborhood convolution kernel is applied:

$$P(\sigma_i, H_j) = \frac{1}{|N(i,j)|} \sum_{(r,c) \in N(i,j)} U(\sigma_r, H_c)$$

P(σi, Hj) = |N(i,j)|1 (r,c)∈N(i,j)∑ U(σr, Hc)

$$N(i,j) = \{(r,c) : \max(|r-i|, |c-j|) \leq 1\}$$

N(i, j) = {(r, c) : max(|r − i|, |c − j|) ≤ 1}

The **Plateau Stability Ratio** ρρ is defined as:

$$\rho(\sigma, H) = \frac{P(\sigma, H)}{U(\sigma, H)}$$

ρ(σ, H) = U(σ, H)P(σ, H)

A cell is classified as a `ROBUST_PLATEAU` if $\rho \geq 0.70$ρ ≥ 0.70, confirming the optimum is supported by neighboring parameter combinations.

3.5.3. Walk-Forward Out-of-Sample (OOS) Cross-Validation

Data is split chronologically into:

- **In-Sample (IS):** First 65% of historical bars ($t = 1, \ \dots, \lfloor 0.65T \rfloor$t = 1, ..., ⌊0.65T⌋).
- **Out-of-Sample (OOS):** Final 35% of historical bars ($t = \lfloor 0.65T \rfloor + 1, \ \dots, T$t = ⌊0.65T⌋ + 1, ..., T).

The optimal (σ^*, H^*)(σ∗, H∗) selected strictly on In-Sample data is tested on untouched Out-of-Sample data.

OOS Efficiency Ratio: $\eta = \frac{WR_{\text{OOS}}}{WR_{\text{IS}}}$η = WRISWROOS

Validation classifications:

- `ROBUST_CONFIRMED`: $\eta \geq 0.70$η ≥ 0.70 and $WR_{\text{OOS}} \geq 45\%$WROOS ≥ 45%.
- `MILD_DEGRADATION`: $0.50 \leq \eta < 0.70$0.50 ≤ η < 0.70.
- `OVERFIT_REJECTED`: $\eta < 0.50$η < 0.50 or $N_{\text{trades, OOS}} < 3$Ntrades, OOS < 3.

3.6. Closed-Form Vectorized Black-Scholes-Merton ATM Option Engine

To enable rapid calculation across multi-thousand parameter iterations without external library latency, the derivative engine implements closed-form Black-Scholes pricing using the Gauss error function $\text{erf}(x)$erf(x).

3.6.1. Analytical Pricing Formula

For an underlying price SS, strike KK, annualized risk-free rate $r = 0.045$r = 0.045, expiration time $T_{\text{years}} = \frac{\text{DTE}}{365}$Tyears = 365DTE, and annualized volatility σσ:

$$d_1 = \frac{\ln(S/K) + (r + \frac{1}{2}\sigma^2)T_{\text{years}}}{\sigma\sqrt{T_{\text{years}}}}, \quad d_2 = d_1 - \sigma\sqrt{T_{\text{years}}}$$

d1 = σ Tyears

ln(S/K) + (r + 21σ2) Tyears, d2 = d1 − σ Tyears

Using the identity between the standard normal cumulative distribution $\Phi(x)$Φ(x) and $\text{erf}(x)$erf(x):

$$\Phi(x) = \frac{1}{\sqrt{2\pi}}\int_{-\infty}^{x} e^{-u^2/2}\,du = \frac{1}{2}[1 + \text{erf}(\frac{x}{\sqrt{2}})]$$

Φ(x) = 2π

1 ∫−∞x e−u2/2du = 21 [1 + erf(2

x)]

The closed-form option prices are:

$$C(S, K, T_{\text{years}}, r, \sigma) = S\Phi(d_1) - Ke^{-rT_{\text{years}}}\Phi(d_2)$$
$$P(S, K, T_{\text{years}}, r, \sigma) = Ke^{-rT_{\text{years}}}[1 - \Phi(d_2)] - S[1 - \Phi(d_1)]$$

C(S, K, Tyears, r, σ)P(S, K, Tyears, r, σ) = SΦ(d1) − Ke−rTyearsΦ(d2) = Ke−rTyears[1 − Φ(d2)] − S[1 − Φ(d1)]

3.6.2. At-The-Money (ATM) Execution Rules

1. **Bullish Trigger ($Z_t \leq -\sigma^*$ Zt ≤ −σ∗):**
   - Direction: `BUY ATM CALL`

- Target Strike: $K = \text{round}(S_{\text{entry}})$K = round(Sentry)

2. **Bearish Trigger ($Z_t \geq +\sigma^*$ Zt ≥ +σ∗):**
   - Direction: `BUY ATM PUT`
   - Target Strike: $K = \text{round}(S_{\text{entry}})$K = round(Sentry)
3. **Volatility Proxy (σ_{entry} σentry):** $\sigma_{\text{entry}} = \max(0.12, \sqrt{252} \times \sqrt{\frac{1}{19}\sum_{k=0}^{19}(u_{t-k} - \bar{u})^2}), \quad u_t = \ln(\frac{S_t}{S_{t-1}})$ σentry = max(0.12, 252 × 191∑k=019(ut−k − u¯)2), ut = ln(St−1St)

3.6.3. P&L and Profit Factor Metrics

For a trade entered at time tt and exited at session $t + H$t + H:

$$P_{\text{entry}} = V(S_t, K, \frac{\tau}{365}, r, \sigma_{\text{entry}})$$

Pentry = V (St, K, 365τ, r, σentry)

$$P_{\text{exit}} = V(S_{t+H}, K, \max(0.001, \frac{\tau - H}{365}), r, \sigma_{\text{entry}})$$

Pexit = V (St+H, K, max(0.001, 365τ − H), r, σentry)

$$\text{ROI}_i = \frac{P_{\text{exit}} - P_{\text{entry}}}{P_{\text{entry}}} \times 100\%$$

ROIi = PentryPexit − Pentry × 100%

⚠ Failed to render LaTeX: KaTeX parse error: Expected 'EOF', got '&' at position 15: \text{Dollar P&L}_i = (P_{\tex...

```
\text{Dollar P&L}_i = (P_{\text{exit}} - P_{\text{entry}}) \times 100
```

⚠ Failed to render LaTeX: KaTeX parse error: Expected 'EOF', got '&' at position 59: ... \text{Dollar P&L}_i > 0} \text...

```
\text{Profit Factor } (PF) = \frac{\sum_{i: \text{Dollar P&L}_i > 0} \text{Dollar P&L}_i}{\sum_{i: \text{Dollar P&L}_i < 0} |\text{Dollar P&L}_i|}
```

3.6.4. Non-Linear Theta Decay Estimation

The expected fractional theta decay over holding period HH calendar days is modeled by the square-root time decay rule:

$$\text{Decay Ratio} = \frac{\tau - \min(\tau - 0.5, H \times \frac{7}{5})}{\tau}$$

Decay Ratio = ττ − min (τ − 0.5, H × 57)

$$\text{Estimated Theta Decay \%} = (1 - \sqrt{\max(0.001, \text{Decay Ratio})}) \times 100\%$$

Estimated Theta Decay % = (1 − max(0.001, Decay Ratio)) × 100%

3.7. Cadence Edge Signal Velocity & Arrival Dynamics

Cadence Edge models trading frequency through an empirical renewal process to deliver predictable cashflow turnover.

3.7.1. Trade Flow Velocity

Let N_i Ni denote the total historical simulated trades executed by income setup ii over an observation window of YY years (typically $Y = 2.7$Y = 2.7 years across 680 trading sessions). The weekly trade arrival rate λ_i λi is:

$$\lambda_i = \frac{N_i}{52 \cdot Y} \quad \text{trades per week}$$

λi = 52 · Y Ni trades per week

3.7.2. Blended Multi-Asset Basket Metrics

For an active user portfolio containing MM selected income setups:

1. Total Weekly Cashflow Frequency: $\Lambda_{\text{portfolio}} = \sum_{i=1}^{M} \lambda_i$ Λportfolio = ∑i=1M λi

2. Weighted Portfolio Win Rate: $\bar{WR} = \frac{\sum_{i=1}^{M} N_i \cdot WR_i}{\sum_{i=1}^{M} N_i}$W R = ∑i=1M Ni∑i=1M Ni·W Ri

3. Weighted Portfolio Holding Duration: $\bar{H} = \frac{\sum_{i=1}^{M} N_i \cdot H_i}{\sum_{i=1}^{M} N_i}$ sessionsH = ∑i=1M Ni∑i=1M Ni·Hi sessions

4. Blended Portfolio Profit Factor: $\bar{PF} = \frac{\sum_{i=1}^{M} N_i \cdot PF_i}{\sum_{i=1}^{M} N_i}$P F = ∑i=1M Ni∑i=1M Ni·P Fi

3.7.3. Real-Time Barrier Proximity Metric

At 3:45 PM EST pre-close, setups are ranked according to standard deviation distance to trigger activation:

$$\delta_{\sigma,i} = \max(0.0, \sigma_i^* - |Z_{t,i}|)$$

δσ,i = max (0.0, σi∗ − |Zt,i|)

- If $\delta_{\sigma,i} = 0.0$δσ,i = 0.0: Signal is actively breaching barrier ($|Z_{t,i}| \geq \sigma_i^*$ |Zt,i| ≥ σi∗).
- If $\delta_{\sigma,i} > 0.0$δσ,i > 0.0: Asset is monitoring within normal volatility bands, ranked by proximity to entry.

4. Data Warehousing & System Infrastructure

The persistence layer is architected as an ACID-compliant local SQL data warehouse using SQLite 3 with normalized schemas, compound primary keys, and conflict resolution mechanisms.

4.1. Core Schema Specifications

1. `stocks`

Master equity registry table defining ticker symbology, market index benchmark associations, and sync status.

- `ticker TEXT PRIMARY KEY`: Stock ticker symbol (e.g. `MSFT`).
- `market TEXT`: Associated market index ETF (e.g. `QQQ`, `SPY`, `DIA`).
- `currency TEXT DEFAULT 'USD'`.
- `last_synced TIMESTAMP DEFAULT CURRENT_TIMESTAMP`.

2. `stock_bars`

Granular daily historical aggregate OHLCV bars.

- Compound Primary Key: `(ticker, trade_date)`.
- Metrics: `open REAL`, `high REAL`, `low REAL`, `close REAL`, `volume INTEGER`, `vwap REAL`.
- Ingestion Rule: `INSERT OR REPLACE INTO stock_bars ...` ensures idempotent updates.

3. `realized_volatility`

Pre-calculated historical volatility features for options pricing.

- Compound Primary Key: `(ticker, trade_date)`.
- Metrics: `log_return REAL`, `rv_10d REAL`, `rv_20d REAL`, `rv_30d REAL`.
- Formulation: $\mathrm{RV}_{N\mathrm{d}} = \sqrt{252} \times \mathrm{std}(\mathrm{log_return}_{[t-N+1:t]})$RVNd = 252

  × std(log_return[t−N+1:t]).

4. `earnings_calendar`

Corporate earnings calendar table used for event-risk filtering.

- Compound Primary Key: `(ticker, earnings_date)`.
- Ingestion Source: Polygon.io `/vX/reference/financials` and corporate disclosure endpoints.

5. `champion_gold_ledger`

Warehouse registry storing the global optimal triplet for each universe stock.

- Primary Key: `ticker TEXT PRIMARY KEY`.
- Fields: `market`, `sector`, `theme`, `threshold`, `contract_dte`, `holding_days`, `profit_factor`, `win_rate_pct`, `avg_roi_pct`, `total_dollar`, `trades_count`, `validation_status`,

`efficiency_ratio`, `oos_win_rate_pct`, `current_z`, `latest_close`, `signal_action`, `is_active`.

6. `champion_gold_trades`

Complete audit log of every historical trade executed by each champion model.

- Primary Key: `id INTEGER PRIMARY KEY AUTOINCREMENT`.
- Fields: `ticker`, `trade_date`, `action`, `entry_underlying`, `strike`, `entry_option_price`, `exit_date`, `exit_underlying`, `exit_option_price`, `pnl_pct`, `dollar_pnl`, `result` (`WIN` or `LOSS`), `failure_reason`.

4.2. Corporate Earnings Announcement Blackout Mechanism

One of the most critical structural components of Project RM is the corporate earnings event filter.

1. **Failure Mode Addressed:** Options entered immediately preceding an earnings release suffer from extreme implied volatility crush post-event, as well as unpredictable gap risk that breaks linear factor models.
2. **Implementation:**
    - Let $E_{\text{dates}} = \{d_1, d_2, \dots, d_K\}$Edates = {d1, d2, ... , dK} be the set of historical earnings announcement dates for an equity.
    - The **Blackout Window** BB includes the announcement date and the immediate following trading session: $B = \bigcup_{k=1}^{K} \{d_k, \text{NextTradingSession}(d_k)\}$B = Uk=1K{dk , NextTradingSession(dk)}
    - Gating: Any signal episode where $\text{EntryDate} \in B$EntryDate ∈ B is blocked from execution: $\text{If } t_{\text{signal}} \in B \implies \text{Status} = \text{BLOCKED_EARNINGS}$If tsignal ∈ B ⟹ Status = BLOCKED_EARNINGS
    - Dossier Edge reports the exact count of filtered trades, verifying the preservation of model edge.

4.3. High-Speed Market Data Ingestion & API Layer

1. **Massive API Client (`api_client.py`):**
    - Primary Provider: Polygon.io REST API.
    - Fallback: Local SQL cache, Yahoo Finance (`yfinance_loader.py`), or synthetic geometric Brownian motion simulator.
    - Resilience: Multi-tier retries with exponential backoff on HTTP 429 rate limits.
2. **Server & Serverless Deployment (`server.py` & `api/index.py`):**

- **Local Environment:** Standalone multi-threaded Python HTTP server (`http.server.HTTPServer`) running on port `8050` with direct SQLite connection pooling.
- **Production Environment (Vercel):** Serverless Python handler (`api/index.py`) routing requests directly to `DashboardHandler`. Endpoints process JSON requests statelessly, reading cached states from optimized SQLite binaries.

5. Verification Checklist for Quantitative Audit

To independently verify the implementation against this mathematical specification, an auditor should check:

Component	Code Location	Key Equations to Verify
Factor Orthogonalization	`residual_model.py` (`analyze`)	$r_{S \perp M} = r_S - \frac{\text{Cov}(r_M, r_S)}{\text{Var}(r_M)} r_M$ rS⊥M = rS − Var(rM)Cov(rM,rS)rM
Kalman Recursion	`residual_model.py` (`_run_filter`)	$e_t = y_t - X_t \beta$et = yt − Xtβ, $S_t = X_t P X_t^T + R$St = XtPXtT + R, $K_t = P X_t^T S_t^{-1}$ Kt = PXtT St−1
QMLE Optimization	`residual_model.py` (`fit`)	$L = \frac{1}{2} \sum (\ln S_t + e_t^2 / S_t)$L = 21 ∑(ln St + et2/St)
OU Half-Life	`residual_model.py` (`OrnsteinUhlenbeckEngine.fit`)	$b = e^{-\theta}$ b = e−θ, $t_{1/2} = \ln(2)/\theta$t1/2 = ln(2)/θ
CUSUM Drift	`residual_model.py` (`CUSUMDriftMonitor.compute`)	$S_t^+ = \max(0, S_{t-1}^+ + Z_t - 0.5)$St+ = max(0, St−1+ + Zt − 0.5), $S_t^- = \max(0, S_{t-1}^- - Z_t - 0.5)$St− = max(0, St−1− − Zt − 0.5)
Semi-Markov Weibull	`semi_markov.py` (`WeibullFit`)	$S(t) = q^{t^\beta}$ S(t) = qtβ, $f(t) = q^{(t-1)^\beta} - q^{t^\beta}$ f(t) = q(t−1)β − qtβ
Option Pricing	`residual_model.py` (`black_scholes_price`)	$\Phi(x) = \frac{1}{2}(1 + \text{erf}(x/\sqrt{2}))$Φ(x) = 21(1 + erf(x/ 2 ⁄)), $C = S\Phi(d_1) - Ke^{-rT}\Phi(d_2)$C = SΦ(d1) − Ke−rTΦ(d2)
Plateau Smoothing	`threshold_optimizer.py` (`_eval_grid`)	$P(\sigma, H) = \text{mean}(U_{3\times3})$P(σ, H) = mean(U3×3), $\rho = P/U$ρ = P/U
Walk-Forward Validation	`threshold_optimizer.py` (`evaluate_contract`	Split at 65%, $\eta = WR_{\text{OOS}}/WR_{\text{IS}}$ η = WROOS/WRIS

Component	Code Location	Key Equations to Verify
	`_tenor_sweep`)	
Earnings Blackout	`residual_model.py` (`evaluate_option_performance`)	Gating on $\{t_{\text{earn}}, t_{\text{earn}} + 1\}${tearn, tearn + 1}
Cadence Proximity	`server.py` (`/api/ledger/cadence-scan`)	$\delta_\sigma = \max(0.0, , \sigma^* -
SQL Warehouse	`warehouse.py` (`_init_schema`)	Table schema definitions and primary key constraints

End of Specification Document.